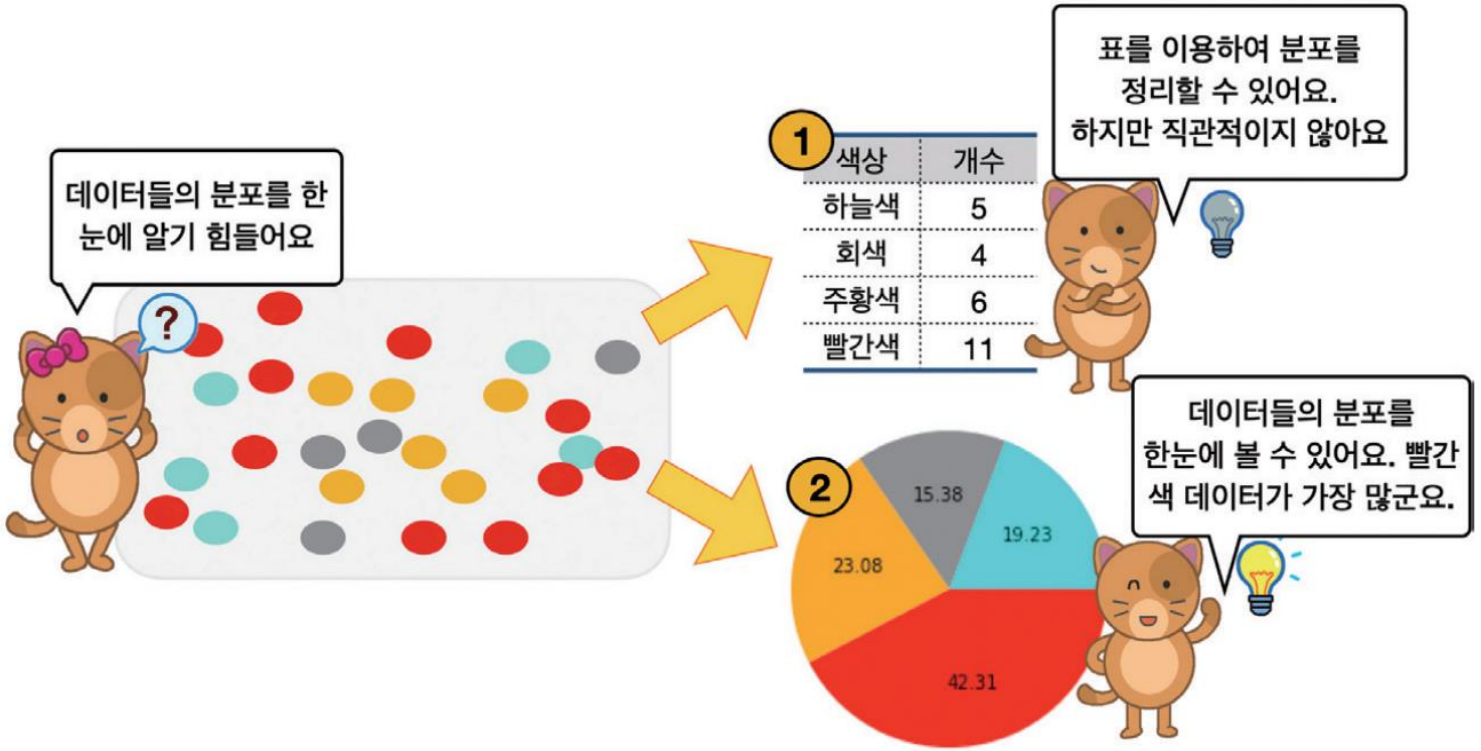
4장 데이터 시각화 도구 맷플롯립

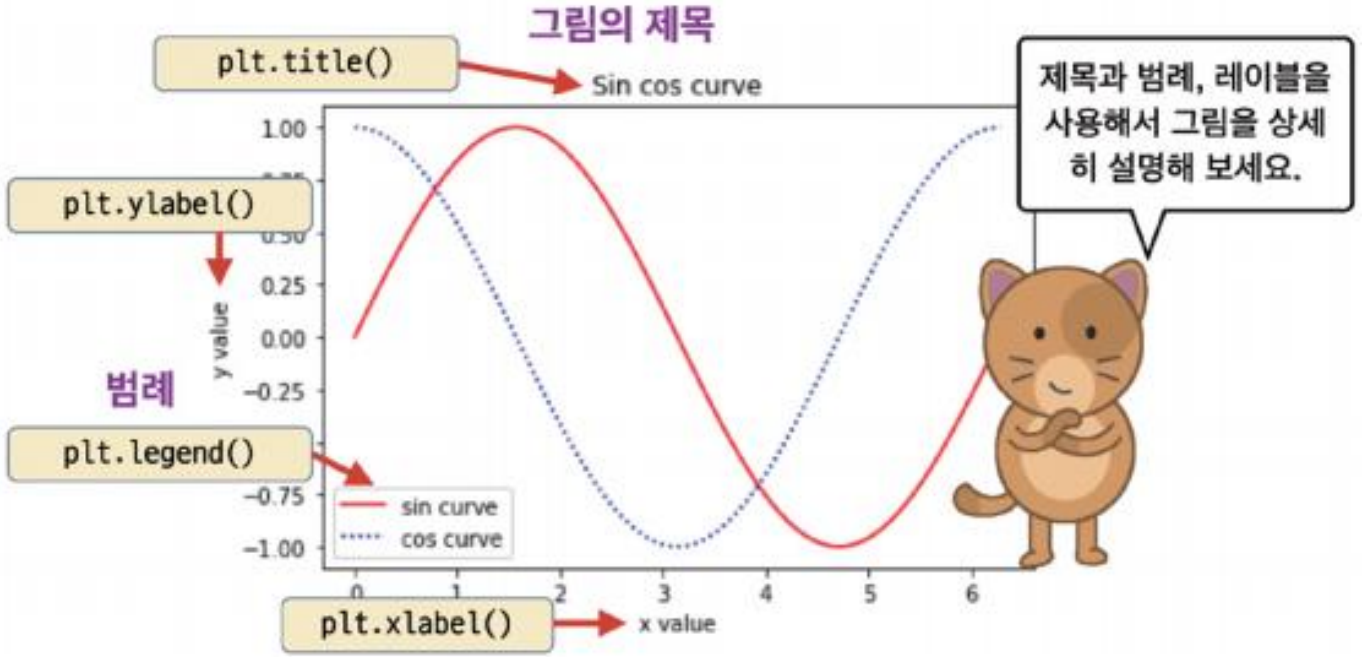
8. 주차별 강의계획(Weekly Schedule)

주 (Week)	강의주제 (Lecture subject)	강의 세부내용 (Detailed contents)
1	- 빅데이터와 인공지능 - 빅데이터 Python 환경 구성	- Git, Python, Visual Studio Code, Anaconda Jupyter notebook, Google Colab
2	- 수치 데이터 처리 : 다차원 배열	- Python 프로그래밍 기초 - Numpy
3	- 통계 데이터의 분석 : 시각화	- matplotlib, seaborn, plotly
4	- 웹크롤링에 의한 데이터 수집과 저장 (교재에 포함 안됨)	- 텍스트파일 : txt, csv, json, PDF, HWP - 공공데이터포탈 API - Naver API - Google Pytrends - BeautifulSoup, Selenium
5	- 텍스트 데이터의 탐색적 분석 : 시각화 (교재에 포함 안됨)	- 워드 클라우딩
6	- 시계열 데이터의 데이터프레임 분석 및 시각화	- Pandas
7	- 데이터 전처리	- 결측 데이터, 중복 데이터, 이상치 처리하기 - 병합 데이터 다루기
8	중간고사	- 필기시험



1. 데이터 과학을 위한 시각화 도구 matplotlib

- **데이터 시각화**은 **컴퓨터 화면에 시각적 이미지를 이용하여 데이터를 보여주는 방법**이며, **데이터를 분석하고 추론하는 데 도움.**
- **matplotlib**은 막대 그래프, 선 그래프, 산점도 등 지원. **Matplotlib.pyplot**에는 **시각화를 위한 함수들과 클래스들이 정의**




```python
import matplotlib as mpl
import matplotlib.pyplot as plt

plt.plot([1, 2, 3, 4])
plt.ylabel('y label')
plt.xlabel('x label')
plt.show()    # 이 코드는 코랩에서는 없어도 됨
```

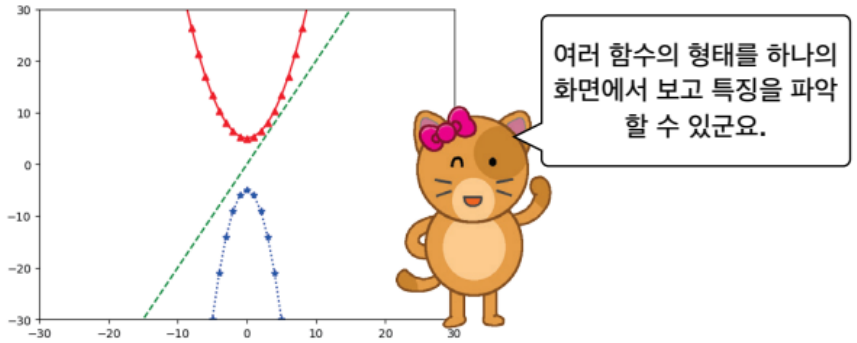
```python
plt.plot([0, 1, 2, 3], [1, 2, 3, 4])
```

```python
x = np.linspace(0, np.pi * 2, 100)

plt.title('Sin cos curve')
plt.plot(x, np.sin(x), 'r-', label='sin curve')
plt.plot(x, np.cos(x), 'b:', label ='cos curve')
plt.xlabel('x value')
plt.ylabel('y value')
plt.legend()
```

1.1 축의 범위(영역)와 그래프의 색상과 모양을 설정하기

- **plt.plot(x, y)** 함수의 입력값으로 **넘파이의 다차원 배열을 사용**.
- **plt.axis()** 함수를 이용하여 **x** 축의 범위와 **y** 축의 범위를 설정.

```python
import matplotlib.pyplot as plt
import numpy as np

x  = np.arange(-20, 20)          # -20에서 20 사이의 수를 1의 간격으로 생성
y1 = 2 * x                       # 2*x를 원소로 가지는 y1 함수
y2 = (1/3) * x ** 2 + 5          # (1/3)*x**2 + 5를 원소로 가지는 y2 함수
y3 = -x ** 2 - 5                 # -x**2 - 5를 원소로 가지는 y3 함수
# 빨강색 점선, 녹색 실선과 세모기호, 파랑색 별표와 점선으로 함수를 표현
plt.plot(x, y1, 'g--', x, y2, 'r^-', x, y3, 'b*:')
plt.axis([-30, 30, -30, 30])     # 그림을 그릴 영역을 지정함
```



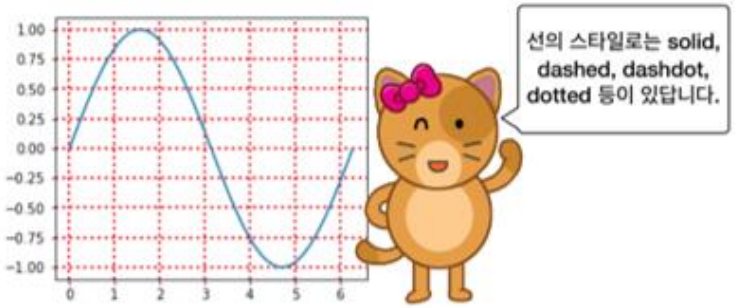
marker	symbol	description
"."	●	point
",".	·	pixel
"o"	●	circle
"v"	▼	triangle_down
"^"	▲	triangle_up
"<"	◀	triangle_left
">"	▶	triangle_right
"1"	Y	tri_down
"2"	⅄	tri_up
"3"	≺	tri_left
"4"	≻	tri_right
"8"	●	octagon
"s"	■	square
"p"	⬟	pentagon
"P"	✚	plus (filled)
"*"	★	star

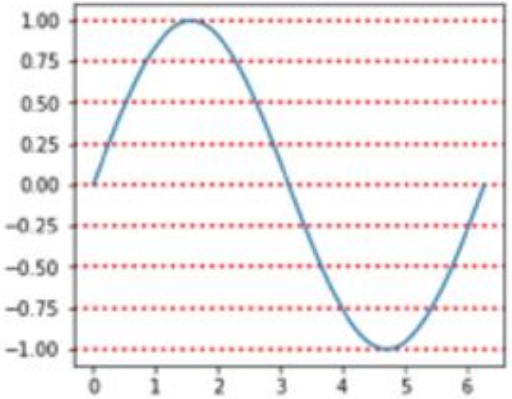
1.2 그래프의 크기와 그리드 그리기

- **plt.figure()**의 키워드 인자 **figsize**은 화면에 나타날 그래프의 크기를 조절.
- **plt.grid()**는 인자 **color, linestyle, linewidth** 등을 사용하여 **격자의 색상, 선의 형태, 선의 굵기를 변경.** axis='y'나 axis='x' 인자를 사용할 경우 수평선, 수직선 격자 선분 추가.

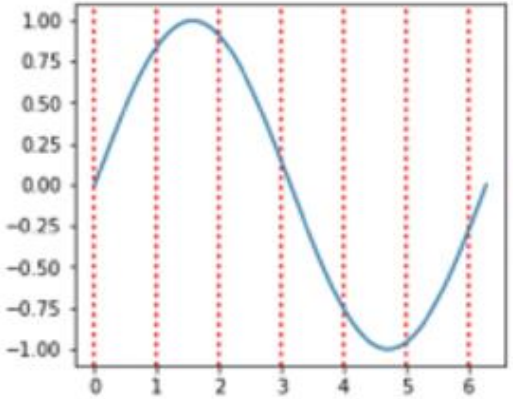
```
X = np.linspace(0, 2*np.pi, 200)
Y = np.sin(X)
plt.figure(figsize=(4.2, 3.6))
plt.plot(X, Y)
```

```
# plt.grid() 대신 다음과 같은 키워드 인자를 지정하여 호출
plt.grid(color='r', linestyle='dotted', linewidth=2)
```





axis='y' 인자를 사용한 경우

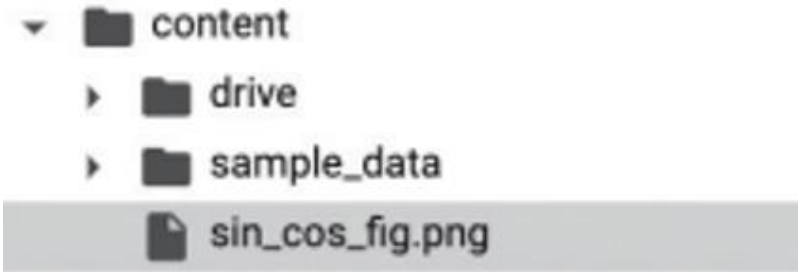


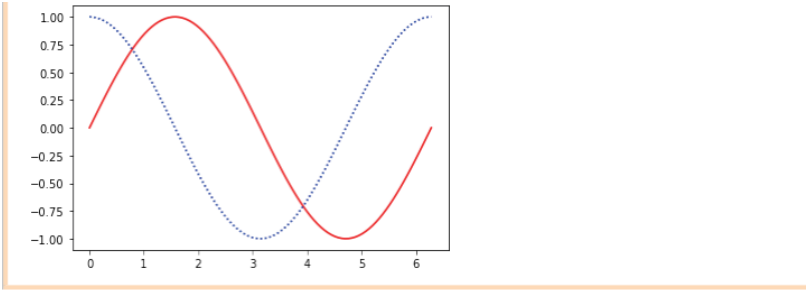
axis='x' 인자를 사용한 경우

1.3 그래프 이미지로 저장하기

- 그림은 **figure** 객체의 **savefig()** 메소드를 사용하여 저장할 수 있다.
  - **plt.figure()**를 통해서 **figure** 객체를 생성하여야 한다.
  - **savefig()** 메소드로 저장 가능한 파일 형식은 **png, jpg, svg, tif, pdf 등**.

```
x = np.linspace(0, np.pi * 2, 100)
fig = plt.figure()
plt.plot(x, np.sin(x), 'r-')
plt.plot(x, np.cos(x), 'b:')

fig.savefig('sin_cos_fig.png')
```





scatter()	산점도 그림을 그리는 기능으로 2차원 좌표에 흩뿌려진 점들의 좌표로 자료값을 나타냄.
bar()	막대 그래프를 그리는 기능, 막대의 모양은 수직 막대.
barh()	수평 막대 그래프를 그리는 기능.
pie()	파이 형태의 그림을 그리는 기능.
imshow()	이미지를 화면에 그리는 기능.
plot()	실선이나 점선으로 데이터를 나타냄.

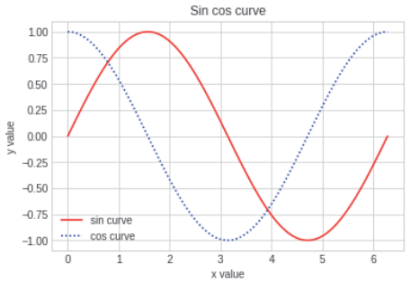
1.4 범주와 스타일에 대해 알아보자

- 범주을 추가하기 위해 **plt.plot()** 메소드의 **label** 인자에 설명글을 추가. **plt.legend()**의 **loc** 인자은 범례의 위치 지정.
- 그림판의 테마 혹은 **스타일시트**을 적용할 때, **plt**의 **style** 객체는 **use()** 메소드를 이용.

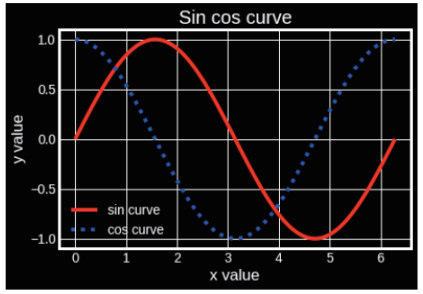
```
plt.legend(loc='upper right')  # 위쪽 오른편에 범례를 표시
```

```
'best', 'upper right', 'upper left', 'lower left', 'lower right', 'right', 'center left', 'center right', 'lower center', 'upper center', 'center'
```

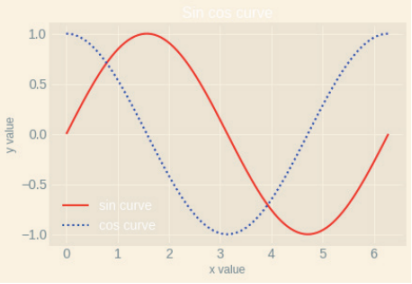
```
# 스타일시트를 변경하는 명령
plt.style.use('seaborn-whitegrid')
```



'seaborn-whitegrid'



'dark_background'



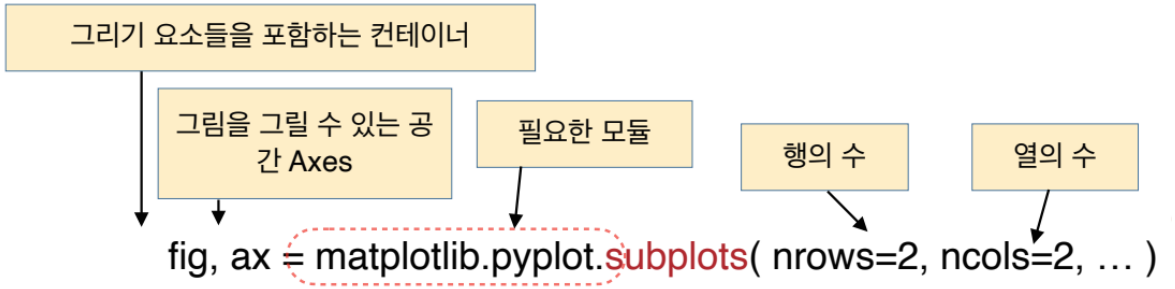
'Solarize_Light2'

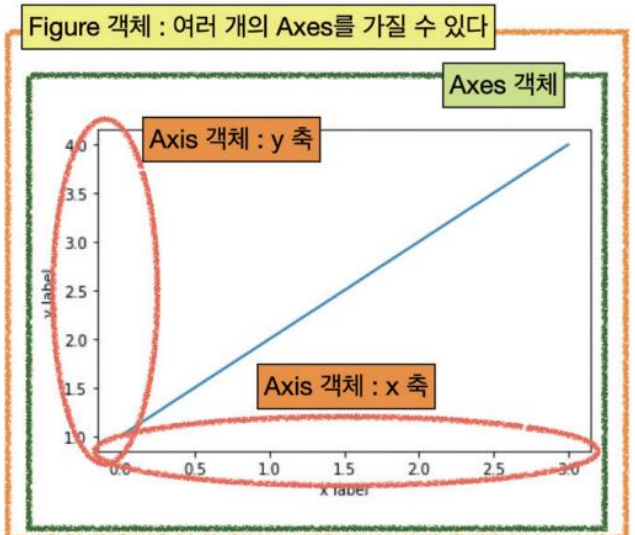
```
# 사용가능한 스타일 목록을 출력하는 명령
print(plt.style.available)
```

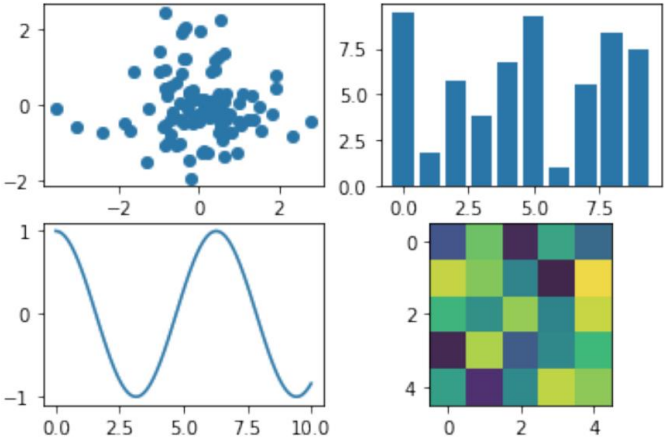
```
['Solarize_Light2', '_classic_test_patch', 'bmh', 'classic', 'dark_background', 'fast', 'fivethirtyeight', 'ggplot', 'grayscale', 'seaborn', 'seaborn-bright', 'seaborn-colorblind', 'seaborn-dark', 'seaborn-dark-palette', 'seaborn-darkgrid', 'seaborn-deep', 'seaborn-muted', 'seaborn-notebook', 'seaborn-paper', 'seaborn-pastel', 'seaborn-poster', 'seaborn-talk', 'seaborn-ticks', 'seaborn-white', 'seaborn-whitegrid', 'tableau-colorblind10']
```

1.5 한 화면에 복수 개 그래프 그리기

- **plt.subplots()을** 사용하여 Figure 객체와 Axes 객체를 반환. **Figure 객체**는 축과 그래픽, 텍스트 등의 **그리기 요소들을 포함하는 컨테이너**, **Axes 객체**는 그림을 그릴 수 있는 테두리 상자 모양의 공간. **Ax는 다차원 배열 객체**이므로 ax[0, 0]과 같은 인덱스를 사용하여 특정한 Axes 객체에 그림을 그림. Axis는 x, y축 객체이니 **Axes와 혼동하지 않도록 함.**





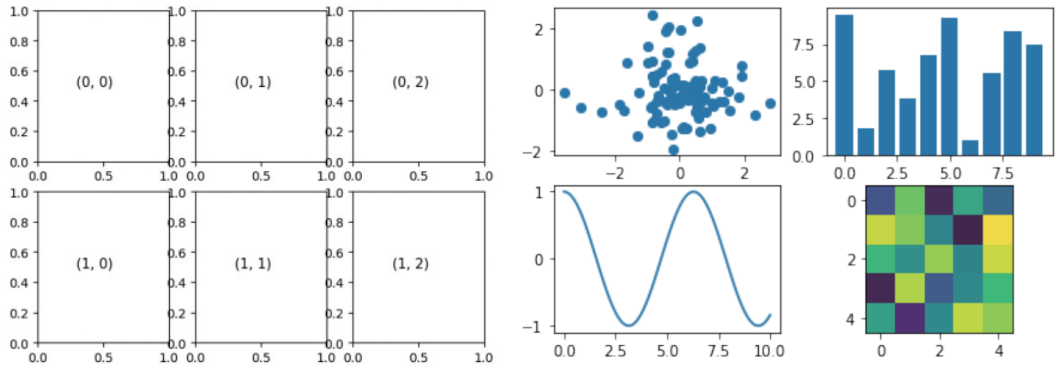


```
fig, ax = plt.subplots(2, 2)

X = np.random.randn(100)
Y = np.random.randn(100)
ax[0, 0].scatter(X, Y)
X = np.arange(10)
Y = np.random.uniform(1, 10, 10)
ax[0, 1].bar(X, Y)
X = np.linspace(0, 10, 100)
Y = np.cos(X)
ax[1, 0].plot(X, Y)
Z = np.random.uniform(0, 1, (5, 5))
ax[1, 1].imshow(Z)
```

1.6 그래프에 텍스트 표시와 그래프 사이의 간격 주기

- plt.subplot()은 plt.subplots() 대신 이용
- wspace 수평 여백을 표시하는 값은 axis의 크기에 대한 상대적인 비율값로 표시. hspace 수직 여백을 표시하는 비율값. 이 값을 **0으로 줄 때 격자 그림 사이의 간격은 사라진다**.

```
fig, ax = plt.subplots(2, 3)

for i in range(2):      # 2행
   for j in range(3):   # 3열
       ax[i, j].text(0.3, 0.5, str((i,j)),fontsize = 11)  # 텍스트만 표시
```



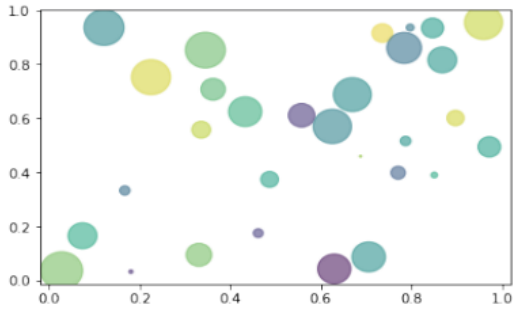

```
grid = plt.GridSpec(2, 3, wspace=0.4, hspace=0.3)
X = np.random.randn(100)  # 정규 분포를 가지는 데이터
Y = np.random.randn(100)  # 정규 분포를 가지는 데이터
```

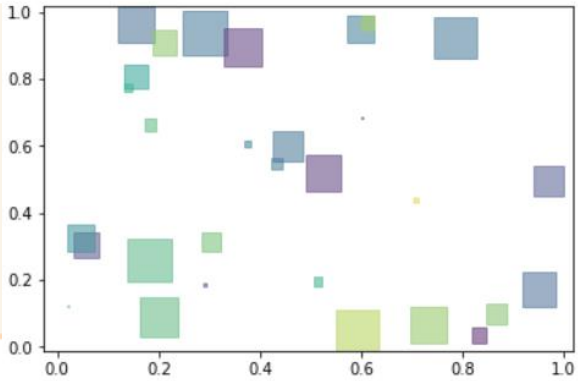
```
plt.subplot(grid[0, 0]).scatter(X, Y)     # 산점도 그림

X = np.arange(10)                         # 0에서 9 사이의 연속값
Y = np.random.uniform(1, 10, 10)          # 균일분포값 생성
# grid[0, 1]과 grid[0, 2]의 2열을 합하여 나타냄
plt.subplot(grid[0, 1:]).bar(X, Y)        # 막대 차트 그림

X = np.linspace(0, 10, 100)
Y = np.cos(X)
plt.subplot(grid[1, :2]).plot(X, Y)       # 실선으로 함수를 그림

Z = np.random.uniform(0, 1, (5, 5))
plt.subplot(grid[1, 2]).imshow(Z)         # 분포를 2D 이미지로 그림
```

2. 자료값의 분포를 나타내는 산점도 그래프

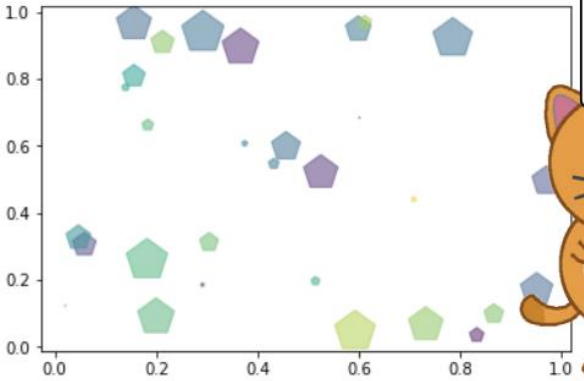
- 산점도는 **두 변수의 상관관계를** 직교 좌표계의 평면에 **데이터를 점으로 표현하는** 그래프.
- **plt.scatter()** 함수의 s 키워드 인자는 점의 면적, c는 점의 색상, alpha는 투명도.
- **marker** 키워드 인자를 넣지 않으면 디폴트 표식은 원, 이 표식을 사각형(r:rectangle), 오각형(p:pentagon) 등의 다양한 모양으로 변경

```
N = 30                                  # 전체 점의 개수
x = np.random.rand(N)                   # 점들의 임의의 x 좌표
y = np.random.rand(N)                   # 점들의 임의의 y 좌표
colors = np.random.rand(N)              # 랜덤한 색상
area = (30 * np.random.rand(N))**2      # 반지름이 0에서 15 포인트 범위

plt.scatter(x, y, s=area, c=colors, alpha=0.5)
```

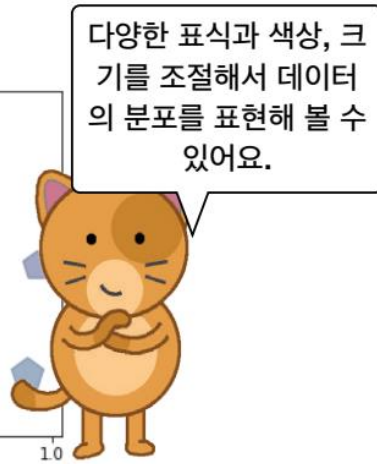




marker = ‘r’



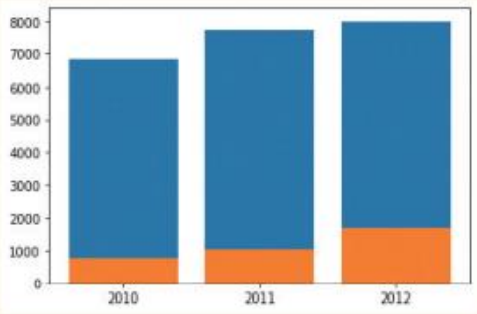
marker = ‘p’



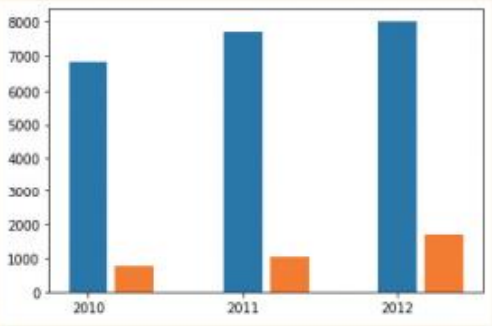
3. 자료값의 분포를 나타내는 막대 그래프

- 막대 그래프는 기본적으로 세로 직사각형 모양으로 데이터값을 나타낼 수 있다.
- **plt.bar()** 함수 사용. **width** 키워드 인자를 통해 막대의 너비를 줄 수 있다. 이 값의 의미는 원래 두께의 25%를 의미하며, x 값에 0.3을 더하여 x 축의 간격을 조절.
- **plt.xticks(x, years)** 를 이용하여 x축으로 연도(years) 눈금값 표시가 가능함.

```
x = np.arange(3)
years = ['2010', '2011', '2012']
domestic = [6801, 7695, 8010]
foreign = [777, 1046, 1681]

plt.bar(x, domestic)
plt.bar(x, foreign)
plt.xticks(x, years)
```



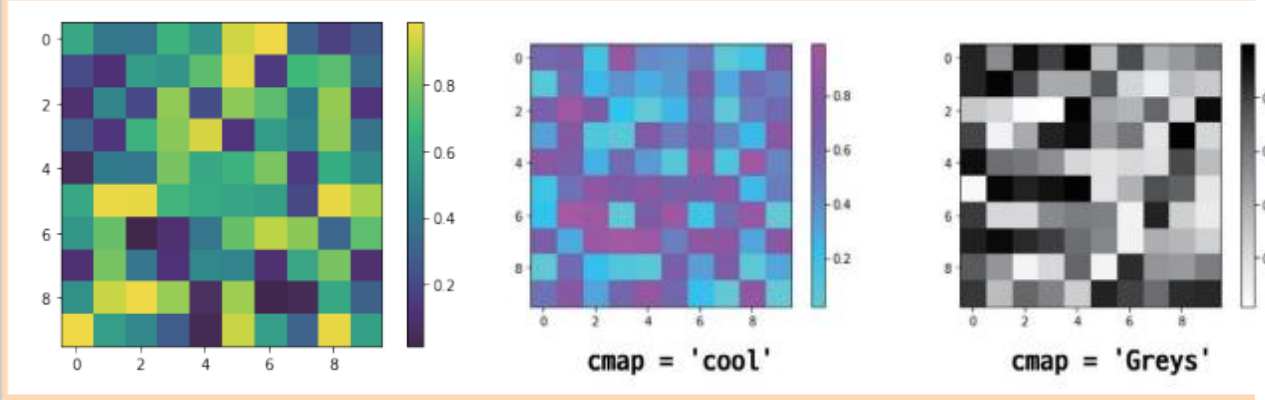

```
x = np.arange(3)
years = ['2010', '2011', '2012']
domestic = [6801, 7695, 8010]
foreign = [777, 1046, 1681]

plt.bar(x, domestic, width=0.25)
plt.bar(x + 0.3, foreign, width=0.25)
plt.xticks(x, years)
```



4. 자료값의 분포를 나타내는 히트맵

- **plt.imshow()은** 이미지 위에 **열 분포 형태로 정보를 표현하는 히트맵.** 데이터의 값이 클 경우와 작은 경우에 대하여 서로 다른 색상을 사용하여 데이터의 분포를 쉽게 이해할 수 있는 유용한 기능. **imshow()** 함수 내의 키워드 인자로 **color map**의 약어인 **cmap**을 넣을 수 있는데, 'cool', 'Greys' 등과 같은 색상맵 타입을 설정.
- **plt.colorbar()**은 그림의 오른쪽에 있는 색상 막대를 생성. 이 색상의 범위가 0에서 1 사이임을 알 수 있다.

```
data = np.random.random((10, 10))  # 임의의 수를 가진 10x10 크기의 배열
plt.imshow(data)
plt.colorbar()
```

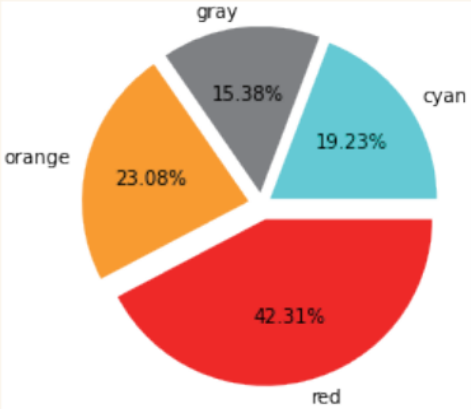


6. 범주형 데이터에 대한 파이 차트

- **plt.pie()은 범주형 데이터**categorical data들이 가지는 전체적인 비율을 파이 모양으로 그림.
- **autopct 인자**는 파이 모양의 부채꼴 안에 표시될 숫자의 형식을 지정하며 **%.2f** 값은 소수점 아래 둘째 자리까지를 표시. 만일 %까지 표시하고자 할 경우 %를 두개 연속해서 입력. **colors** 인자는 각 섹션의 색상을 지정하는 역할을 하는데 이를 생략하면 디폴트 색상을 부여. **explode** 인자는 각 항목이 파이의 원점에서 튀어나오는 정도를 표시.

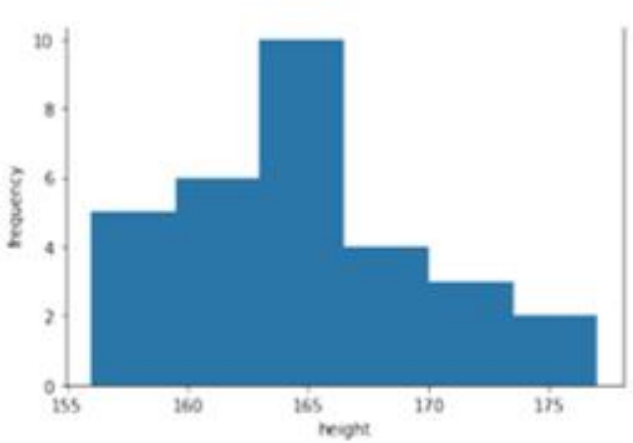
색상	개수
하늘색	5
회색	4
주황색	6
빨간색	11

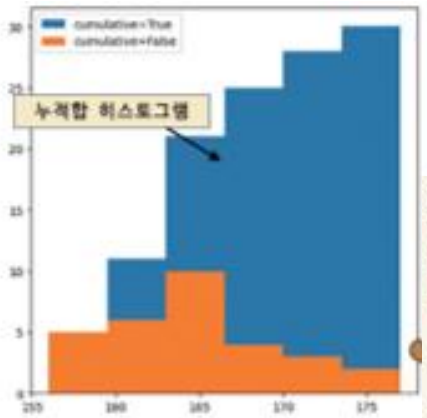
```
import matplotlib.pyplot as plt
import numpy as np

data = [5, 4, 6, 11]
clist = ['cyan', 'gray','orange', 'red']
explode = [.06, .07, .08, .09]
plt.pie(data, autopct = '%.2f%%',
        colors=clist, labels=clist, explode=explode)
```



6. 자료값의 분포를 나타내는 히스토그램

- **히스토그램**histogram은 주어진 자료를 몇 개의 구간으로 나누고 각 구간의 **도수**frequency를 조사하여 나타낸 막대 그래프. 히스토그램의 **가로 축은 구간, 세로 축은 도수를** 표시. 데이터 분포를 지정된 구간에 나타내기위해 데이터의 값을 동일한 구간으로 나눠야 한다. 데이터가 들어가는 통을 **빈**bin.
- **plt.hist()의 bins** 인자 사용하여 빈을 표시, **cumulative 인자는** 누적된 자료값의 합을 출력하기 위해 사용, 데이터의 개수가 30이므로 가장 오른쪽의 히스토그램 높이가 30인 것도 확인.





cumulative 인자가 True이면 자료의 누적합을 구해서 그려주네요.

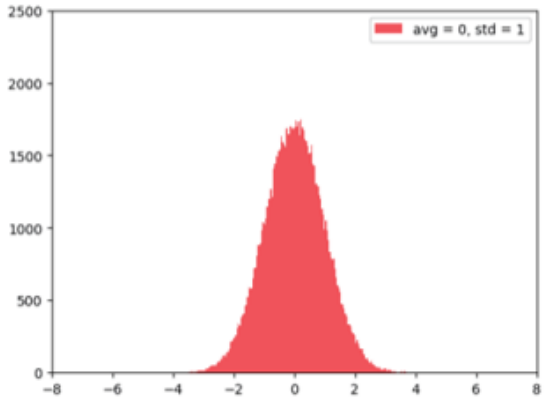
```
heights = np.array([175, 165, 164, 164, 171, 165, 160, 169, 164, 159, 163,
167, 163,172, 159, 160, 156, 162, 166, 162, 158, 167, 160, 161, 156, 172, 168,
165, 165, 177])
plt.hist(heights, bins=6)      # 히스토그램 bins는 구간의 수
plt.xlabel("height")
plt.ylabel("frequency")
```

```
plt.hist(heights, bins=6, label='cumulative=True', cumulative=True)
plt.hist(heights, bins=6, label='cumulative=False', cumulative=False)
plt.legend(loc='upper left')
```

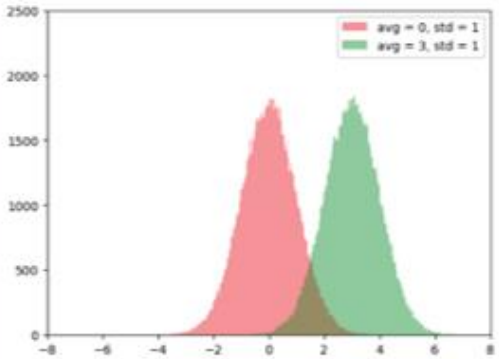
7. 히스토그램을 이용한 정규 분포(normal distribution) 함수 그리기

- **표준 정규 분포란 연속 확률 분포**의 하나이며 **평균이 0이고 분산이 1**인 정규 분포.
- 표준 정규 분포는 **종모양의 분포, 평균값 0 근방이 가장 높고 좌우로 갈수록 낮아지는 대칭** 형태.
- **random.normal()을** 이용하여 인자 loc와 scale을 이용하여 평균과 표준 편차를 설정. 종모양의 분포도에서 가장 높은 곳의 x축 값이 평균값. 표준 편차가 커질수록 종모양의 그래프가 넓게 퍼짐.

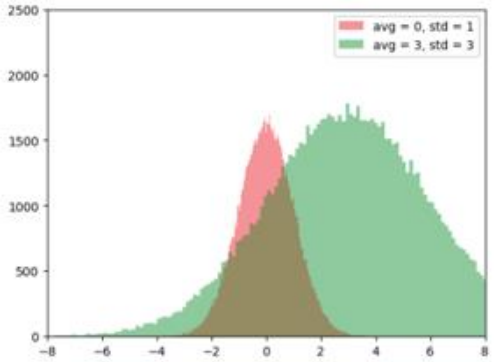
```
import numpy as np
import matplotlib.pyplot as plt

f1 = np.random.randn(100000)  # 10만 개의 난수 생성
plt.hist(f1, bins=200, color='red', alpha=.7,\
         label='avg = 0, std = 1')
plt.axis([-8, 8, -2, 2500])
plt.legend()
```




```
f1 = np.random.normal(loc=0, scale=1, size=100000)
f2 = np.random.normal(loc=3, scale=1, size=100000)

plt.hist(f1, bins=200, color='red', alpha=.4,
         label='avg = 0, std = 1')
plt.hist(f2, bins=200, color='green', alpha=.4,\
         label='avg = 3, std = 1')
plt.axis([-8, 8, -2, 2500])
plt.legend()
```




```
f1 = np.random.normal(loc=0, scale=1, size=100000)
f2 = np.random.normal(loc=3, scale=3, size=100000)

plt.hist(f1, bins=200, color='red', alpha=.4,
         label='avg = 0, std = 1')
plt.hist(f2, bins=200, color='green', alpha=.4,
         label='avg = 3, std = 3')
plt.axis([-8, 8, -2, 2500])
plt.legend()
```



8. 정규 분포 함수와 확률 밀도 함수 그리기

- **scipy 모듈**은 과학계산을 위해 넘파이, 맷플롯립, 판다스과 연계함. **scipy.stats.norm** 클래스의 **rvs()**는 random value sampling의 약자. **무작위로 정규분포 표본** 반환, loc 인자는 분포의 기대값, scale 인자는 분포의 표준 편차를 의미.
- plt.hist()의 density=True인 경우 norm.fit()의 **평균과 표준편차**로 norm.pdf()의 **확률 밀도 함수 그래프을** 히스토그램에 겹쳐 그림.

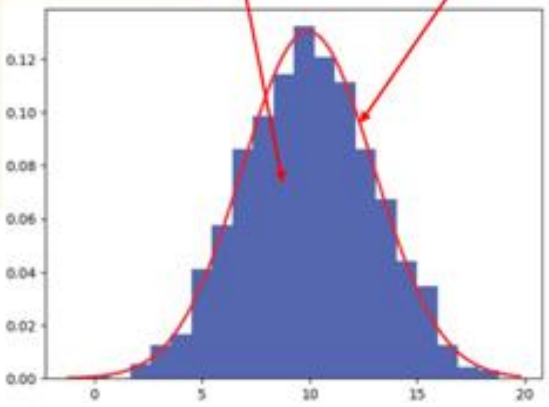
```
from scipy.stats import norm

# 평균이 10.0이고 분산이 3인 1,000개의 데이터를 생성
data = norm.rvs(10.0, 3, size=1000)
# 히스토그램을 그림
plt.hist(data, bins=20, density=True, alpha=0.6, color='b')

# 데이터를 이용하여 평균과 표준편차를 구함
mu, std = norm.fit(data)
# scipy의 pdf는 평균, 표준편차로부터 확률 밀도 함수를 생성한다
xmin, xmax = plt.xlim()  # x의 최소, 최대값을 pyplot으로부터 구함
x = np.linspace(xmin, xmax, 100)
p = norm.pdf(x, mu, std)
plt.plot(x, p, 'r', linewidth=2)
```

scipy 패키지의 stat 서브 패키지에 있는 norm 모듈을 사용하여 정규 분포 데이터를 생성하자.

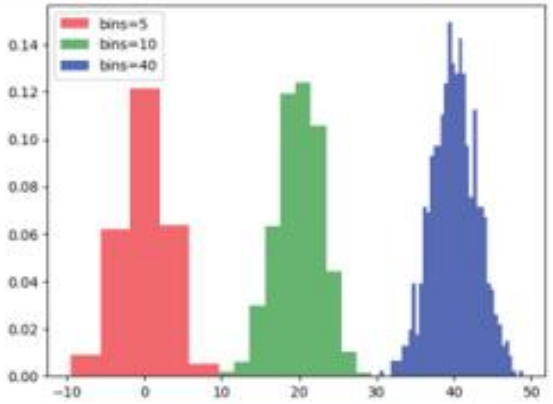
파란색 막대가 정규 분포를 가지는 데이터

빨간 실선이 확률 밀도 함수임

빨간색 실선으로 그려지는 확률 밀도(pdf) 함수

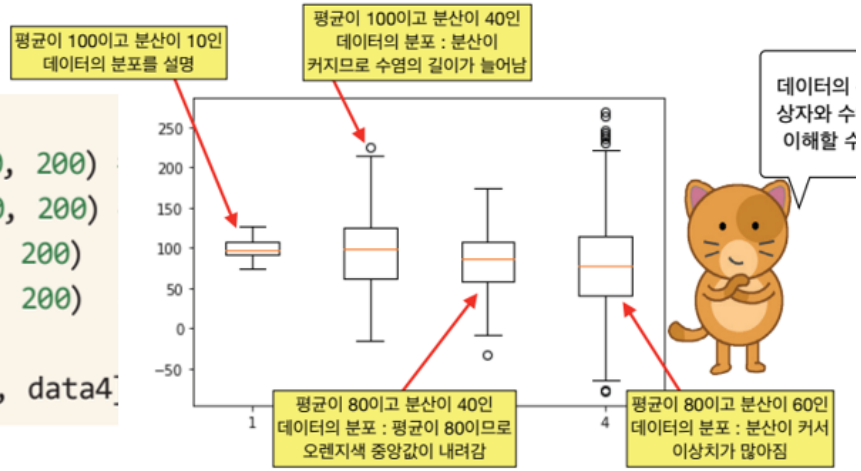
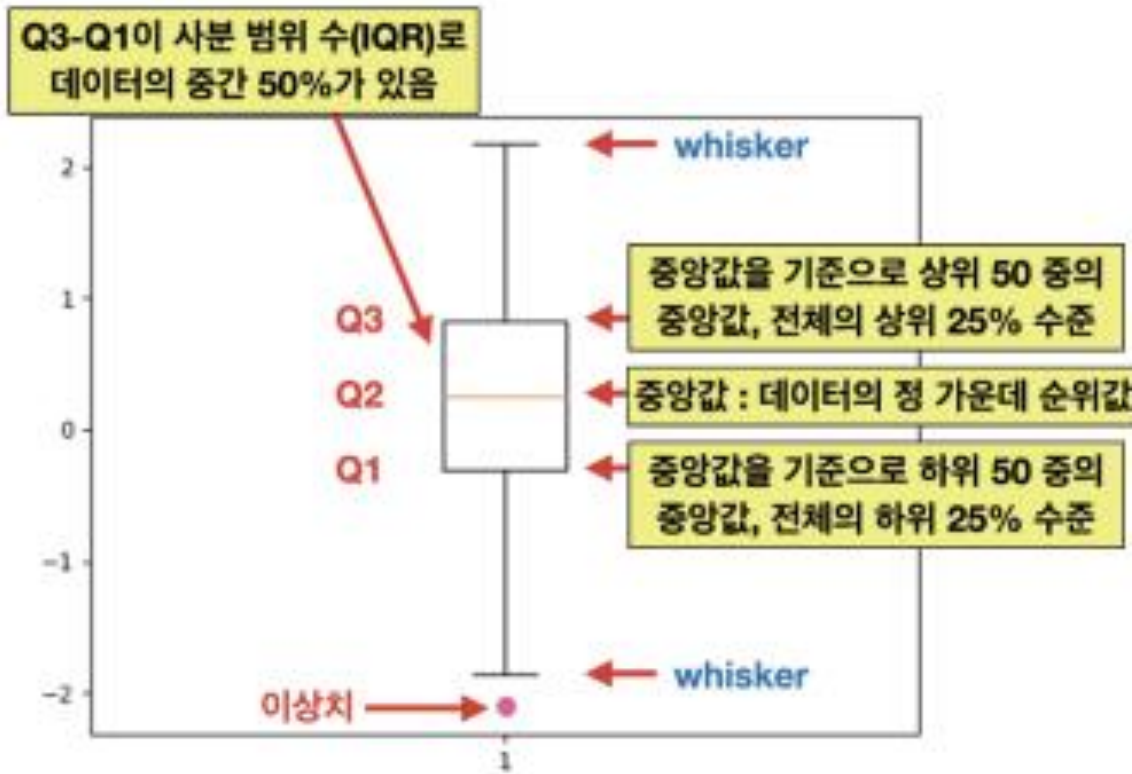



```
from scipy.stats import norm

# 평균이 0.0이고 분산이 3인 1,000개의 데이터를 생성
data = norm.rvs(0.0, 3, size=1000)
# 5개 구간에 넣어서 히스토그램을 그리자
plt.hist(data, bins=5, density=True, alpha=0.6, color='r',\
         label = 'bins=5')
# 평균이 20.0이고 분산이 3인 1,000개의 데이터를 생성
data = norm.rvs(20.0, 3, size=1000)
# 10개 구간에 넣어서 히스토그램을 그리자
plt.hist(data, bins=10, density=True, alpha=0.6, color='g',\
         label = 'bins=10')
# 평균이 40.0이고 분산이 3인 1,000개의 데이터를 생성
data = norm.rvs(40.0, 3, size=1000)
# 40개 구간에 넣어서 히스토그램을 그리자
plt.hist(data, bins=40, density=True, alpha=0.6, color='b',\
         label = 'bins=40')
plt.legend()
```



9. 자료값의 분포를 나타내는 상자 수염(box-and-whisker plot, box plot)

- **최대, 최소, 중간값과 사분위 수의 통계량을 하나의 상자 그림으로 표시**. 상자 모양 직사각형은 **중앙값** Q2를 중심으로 상위 25%(Q3)와 하위 25%(Q1)의 값이 모여 있는 구간. Q3-Q1을 **사분 범위** 혹은 **IQR**inter-quartile range. 상자 아래위로 직선이 그어져 있고 이를 위와 아래에서 막는 선이 있는데 이것을 **위스커**whisker 수염. 수염의 끝에 막대를 그어 **이상치** 자료 값의 경계를 표시. 즉 아래쪽 위스커에서 위쪽 위스커까지의 범위가 대부분의 데이터가 분포하는 범위.

- 상단 수염: Q3 + 1.5 * IQR 값보다 작은 데이터 중 가장 큰 값
- 하단 수염: Q1 − 1.5 * IQR 값보다 큰 데이터 중 가장 작은 값

```
np.random.seed(85)
data1 = np.random.normal(100, 10, 200)
data2 = np.random.normal(100, 40, 200)
data3 = np.random.normal(80, 40, 200)
data4 = np.random.normal(80, 60, 200)

plt.boxplot([data1, data2, data3, data4]
```